

MOBILE APPLICATION AND DEVELOPMENT • WEEK 3 • THEORY 2

Receivers, Providers & the Manifest

Broadcast Receivers • Content Providers • AndroidManifest.xml • Permissions

Ekramul Haque Amin • Lecturer, AI & Data Science • Dhaka International University

TODAY'S ROADMAP

What We Will Cover

- 1 Quick recap: Activities and Services**
- 2 Broadcast Receivers: reacting to events**
- 3 Content Providers: sharing data between apps**
- 4 AndroidManifest.xml: how the system learns about your app**
- 5 Permissions and project structure**

LAST CLASS

Recap: Activities and Services

Activity

One screen. onCreate, onStart, onResume, onPause, onStop, onDestroy.

Service

Background work with no UI. Started, bound or foreground.

Intent

The message that starts an Activity or a Service.

Today we finish the other two components, then look closely at the file that declares all four.

PART 1 – BROADCAST RECEIVERS

What Is a Broadcast Receiver?

A Broadcast Receiver lets the system and apps send and receive broadcast messages, like publish-subscribe. When an event happens Android broadcasts an Intent, and every registered receiver is notified, even if the app is not currently running.

Publisher

The system or an app sends a broadcast Intent

Subscriber

Your receiver declares interest in that action

Reaction

onReceive() runs and responds

PUBLISH AND SUBSCRIBE

How a Broadcast Travels

```
Sender (system or your app)
  | sendBroadcast(intent)
  v
Android system -- matches the action → registered receivers
                                     |
                                     Receiver A.onReceive()
                                     Receiver B.onReceive()
```

The sender does not know who is listening. Each receiver decides for itself what to do.

TWO WAYS TO REGISTER

Static vs. Dynamic Receivers

Static (manifest)

Declared in a `<receiver>` tag in `AndroidManifest.xml`. Can react even when the app is not running, within modern background limits.

Dynamic (context)

Registered at runtime with `registerReceiver()` and removed with `unregisterReceiver()`. Active only while the registering component is alive.

Dynamic registration is now the default choice for most broadcasts.

EVENTS ANDROID ANNOUNCES

Common System Broadcasts

Action	Meaning	Note
ACTION_BOOT_COMPLETED	Device finished booting	Needs RECEIVE_BOOT_COMPLETED
ACTION_BATTERY_LOW	Battery level is low	
ACTION_AIRPLANE_MODE_CHANGED	Airplane mode was toggled	
ACTION_POWER_CONNECTED	Charger was plugged in	

REACTING TO A BROADCAST (JAVA)

AirplaneModeReceiver

```
public class AirplaneModeReceiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        boolean on = intent.getBooleanExtra("state", false);  
        Toast.makeText(context,  
            on ? "Airplane mode ON" : "Airplane mode OFF",  
            Toast.LENGTH_SHORT).show();  
    }  
}
```

The Intent carries the details. Here the extra named state says whether airplane mode is on.

STATIC REGISTRATION

Declaring a Receiver in the Manifest

```
<receiver
    android:name=".BootReceiver"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED" />
    </intent-filter>
</receiver>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
```

BOOT_COMPLETED is one of the few system broadcasts that Android still delivers to manifest receivers.

REGISTER AND UNREGISTER IN PAIRS

Dynamic Registration

```
private final AirplaneModeReceiver receiver = new AirplaneModeReceiver();

@Override protected void onStart() {
    super.onStart();
    registerReceiver(receiver,
        new IntentFilter(Intent.ACTION_AIRPLANE_MODE_CHANGED));
}

@Override protected void onStop() {
    super.onStop();
    unregisterReceiver(receiver);
}
```

Match `onStart` with `onStop`, or `onResume` with `onPause`, so the receiver never outlives the screen.

YOUR OWN EVENTS

Sending a Custom Broadcast

```
// Sender
Intent i = new Intent("com.example.app.ORDER_PLACED");
i.setPackage(getPackageName()); // keep it inside this app
i.putExtra("orderId", 42);
sendBroadcast(i);

// Inside the receiver
int id = intent.getIntExtra("orderId", -1);
```

Use a unique action string, usually your package name plus an event name.

RULES KEEP CHANGING

Modern Android Restrictions

Android 7.0+

CONNECTIVITY_CHANGE no longer reaches manifest receivers.

Android 8.0+

Most implicit broadcasts cannot be declared in the manifest. Register dynamically.

Android 14

Apps targeting it must pass **RECEIVER_EXPORTED** or **RECEIVER_NOT_EXPORTED** when registering.

For network changes use `ConnectivityManager.NetworkCallback`. `onReceive()` must finish in about 10 seconds, so hand long work to a Service.

PART 2 – CONTENT PROVIDERS

What Is a Content Provider?

A Content Provider manages access to a structured set of data (a SQLite database, files or network data) and exposes it through a standard interface so other apps can query, insert, update or delete, subject to permissions.

Contacts

Names, numbers, emails

Calendar

Events and reminders

Media

Photos, audio and video

WHY BOTHER?

Why Use a Content Provider?

- 1 A consistent, permission-controlled way to share data between apps**
- 2 It hides the storage mechanism from the components that consume the data**
- 3 It works with content: URIs used by ListView/RecyclerView adapters and file pickers**

If data is only used inside your own app, you do not need a Content Provider. Use SQLite or Room directly.

ONE INTERFACE ON BOTH SIDES

How a Provider Is Reached

Client app --ContentResolver→ ContentProvider → SQLite / files

query() insert() update() delete()

(the same four calls on the client and on the provider)

The client never opens your database. It only talks to the provider through a ContentResolver.

ADDRESSING DATA

Content URIs

`content://<authority>/<path>/<optional id>`

`content://com.android.contacts/contacts/5`

content://
Scheme: this is provider data

com.android.contacts
Authority: which provider

contacts/5
Path and id: which row

READING THE CONTACTS PROVIDER (JAVA)

Querying a Content Provider

```
Cursor cursor = getContentResolver().query(
    ContactsContract.Contacts.CONTENT_URI,
    null, null, null, null);
if (cursor != null) {
    while (cursor.moveToNext()) {
        String name = cursor.getString(cursor.getColumnIndexOrThrow(
            ContactsContract.Contacts.DISPLAY_NAME));
    }
    cursor.close();
}
```

Reading contacts needs the `READ_CONTACTS` permission, requested at runtime on modern Android.

FROM THE CLIENT SIDE

Insert, Update and Delete

```
// NotesContract.CONTENT_URI belongs to your own provider
ContentValues v = new ContentValues();
v.put("title", "Buy milk");
Uri row = getContentResolver().insert(NotesContract.CONTENT_URI, v);

v.put("title", "Buy bread");
getContentResolver().update(row, v, null, null);
getContentResolver().delete(row, null, null);
```

`insert()` returns the Uri of the new row. Reuse that Uri to update or delete it.

BUILDING YOUR OWN PROVIDER

The Six Methods You Implement

Method	Called when	You return
onCreate()	Provider starts	true, after opening the DB
query()	A client reads	A Cursor
insert()	A client adds a row	The new row's Uri
update()	A client changes rows	Number of rows changed
delete()	A client removes rows	Number of rows deleted
getType()	A client asks the type	A MIME type string

A PROVIDER IS ONE CLASS

NotesProvider Skeleton

```
public class NotesProvider extends ContentProvider {  
    @Override public boolean onCreate() { /* open DB */ return true; }  
    @Override public Cursor query(Uri uri, String[] projection, String selection,  
                                   String[] args, String sortOrder) { /* ... */ }  
    @Override public Uri insert(Uri uri, ContentValues values) { /* ... */ }  
    @Override public int update(Uri u, ContentValues v, String s, String[] a) { /* ... */ }  
    @Override public int delete(Uri u, String s, String[] a) { /* ... */ }  
    @Override public String getType(Uri uri) { /* ... */ }  
}
```

REGISTER IT, THEN SHARE IT

Declaring a Provider

```
<provider  
    android:name=".NotesProvider"  
    android:authorities="com.example.notes.provider"  
    android:exported="false" />
```

The authority must be unique on the device. It becomes the host part of every content URI.

ALREADY ON EVERY PHONE

Built-in Providers You Can Use

Provider class	Data	Permission
ContactsContract	Contacts, numbers, emails	READ_CONTACTS
CalendarContract	Calendar events	READ_CALENDAR
MediaStore	Photos, audio and video	READ_MEDIA_IMAGES on Android 13+
CallLog	Call history	READ_CALL_LOG

PART 3 – ANDROIDMANIFEST.XML

What Is the Manifest?

The manifest is your app's ID card for the system. Before running any of your code, Android reads it to learn what the app contains and what it is allowed to do.

Components

Every Activity, Service, Receiver and Provider

Permissions

What the app asks the user to allow

Requirements

Hardware and software features needed

Identity

Icon, label and theme of the app

THE SKELETON

Anatomy of a Manifest

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-feature android:name="android.hardware.camera"
    android:required="false" />
  <application
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/Theme.MyApp">
    <activity android:name=".MainActivity" android:exported="true" />
  </application>
</manifest>
```

THE TAGS YOU WILL MEET

Key Manifest Elements

Element	Purpose	Example
<uses-permission>	Asks for a permission	INTERNET, CAMERA
<uses-feature>	Declares hardware needs	camera, GPS
<application>	App-wide settings	icon, label, theme
<activity> <service>	Declare components	one tag each
<receiver> <provider>	Declare the other two	same idea
<intent-filter>	What a component handles	action, category, data

TELLING ANDROID WHAT YOU CAN HANDLE

Intent Filters

```
<activity android:name=".ShareActivity" android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.SEND" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:mimeType="text/plain" />
  </intent-filter>
</activity>
```

An implicit Intent matches a component when its action, category and data all fit the filter.

WHO MAY START THIS COMPONENT?

The exported Attribute

exported = true

Other apps can start this component. Needed for the launcher Activity.

exported = false

Only your own app can start it. The safe default.

Apps targeting Android 12 (API 31) or higher must set `android:exported` on every component that has an intent-filter.

THREE KINDS

Permission Types

Normal

Granted automatically at install.

Example: INTERNET.

Dangerous (runtime)

The user must approve while the app runs. Examples: CAMERA, READ_CONTACTS, LOCATION.

Special

Granted from a system settings screen.

Example: drawing over other apps.

Dangerous permissions must be both listed in the manifest and requested at runtime.

ASK AT THE MOMENT YOU NEED IT

Requesting a Permission at Runtime

```
if (ContextCompat.checkSelfPermission(this, Manifest.permission.READ_CONTACTS)
    ≠ PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
        new String[]{Manifest.permission.READ_CONTACTS}, 100);
}
@Override
public void onRequestPermissionsResult(int code, String[] perms, int[] results) {
    super.onRequestPermissionsResult(code, perms, results);
    if (code == 100 && results[0] == PackageManager.PERMISSION_GRANTED) loadContacts();
}
```

WHERE THINGS LIVE

Android Studio Project Structure

manifests

AndroidManifest.xml, the file we just studied

java

Your source code, such as MainActivity

res

layout, drawable, mipmap and values (strings, colors, themes)

Gradle Scripts

build.gradle and settings.gradle control the build

NOT EVERYTHING IS IN THE MANIFEST

Module build.gradle Essentials

```
android {  
    namespace 'com.example.myapp'  
    compileSdk 34  
    defaultConfig {  
        applicationId "com.example.myapp"  
        minSdk 24  
        targetSdk 34  
        versionCode 1  
        versionName "1.0"  
    }  
}
```

minSdk is the oldest Android you support. **targetSdk** is the version you have tested against. Numbers here are examples.

SIDE BY SIDE

The Four Components Together

Need to...	Use	Declared as
Show a screen	Activity	<activity>
Work with no UI	Service	<service>
React to an event	Broadcast Receiver	<receiver> or at runtime
Share app data	Content Provider	<provider>

LEARN FROM THESE NOW

Common Mistakes

Missing manifest entry

Component not declared, so it never starts

Heavy work in onReceive

The receiver gets killed after a few seconds

Skipping the permission check

A SecurityException crashes the app

Not unregistering

Dynamic receivers leak the Activity

IN-CLASS EXERCISE, 15 MINUTES

Receiver and Permission

- 1 Create `AirplaneModeReceiver` and show a Toast in `onReceive()`
- 2 Register it in `onStart()` and unregister it in `onStop()`
- 3 Toggle airplane mode from the emulator's quick settings and read the Toast
- 4 Add `READ_CONTACTS` to the manifest and request it at runtime
- 5 Log the name of the first contact once permission is granted

BEFORE YOU LEAVE TODAY

Recap Checklist

- 1 Can you explain static vs. dynamic Broadcast Receivers?
- 2 Do you know why a Content Provider needs an authority and a URI?
- 3 Can you name the six methods of a ContentProvider?
- 4 Do you know what exported="true" allows?
- 5 Can you request a dangerous permission at runtime?

WHERE WE GO FROM HERE

Next Steps

This week: Lab

Components walkthrough demo: lifecycle, a started Service and a receiver

Week 4

Android UI layouts: LinearLayout, RelativeLayout and ConstraintLayout

Before the lab, make sure your project builds and your emulator is running.

Questions?

Receiver

Provider

Manifest

Permissions

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)